Java Reflection

1. What is Reflection?

→ This is used to examin the classes, Methods, fields, interfaces at runtime and also possible to change the behaviour of class too. for example:

- what all methods present in the class
- what all fields present in the class.
- what is the return type of the method.
- what is the modifier of the class.
- what all interfaces class has implemented.
- change the value of the public and private field of the class etc.

2. How to do Reflection of classes?

→ To reflect the class, we first need to get an object of class.

(So, lets first understand, Class then we will come back to how to reflect the class.)

What is this class Class?

- Instance of the class Class represents classes during runtime.
- JVM creates one Class object for each and every class which is loaded during runtime.
- This Class object, has meta data information about the particular class like its method, fields, constructor etc.

class Bird { —then JVM creates at runtime→ class Class { // contain metadata of // Bird class }

}

How to get the particular class Class object?

→ There are 3 ways.

1. Using forName() method :

```
//assume that we have one class called Bird
class Bird { }

// get the object of Class for getting the
// metadata information of Bird class.
    Class birdClass = Class.forName("Bird");
```

2. Using .class :

```
class Bird { }
// get the object of Class for getting the metadata
// information of Bird class.

Class birdClass = Bird.class;
```

3. Using getClass() method :

```
class Bird { }

Bird birdObj = new Bird();
// get the object of class for getting the metadata
// information of Bird class.

Class birdClass = birdObj.getClass();
```

Now understand, how to do Reflection of Classes.

ex:

```
public class Eagle {
    public String breed;
    private boolean canSwim;

    public void fly(){
        sout("fly");
    }
    public void eat(){
        sout("eat");
    }
}
public class Main{
    psvm(){
        Class eagleClass = Eagle.class; // get Class obj of Eagle class
        sout(eagleClass.getName()); //o/p: Eagle
/* o/p = public */  sout(Modifier.toString(eagleClass.getModifiers()));
```

output: Eagle
Public

Note: Methods Available in Class object, all are getters, not Set methods.

- getClassLoader()
- asSubclass(), getMethods()
- cast(), getDeclaredMethods(), getFields()
- getClass(), getConstructors(), getDeclaredFields()

etc.

* The Package "java.lang.reflect" provide classes that can be used to acess and manipulate fields, methods, constructors etc.

Reflection of Methods :

```
public class Eagle {
    public String breed;
    private boolean canSwim;
    public void fly() {
        sout("fly");
    }
    private void eat() {
        sout("eat");
    }
}

public class Main {
    psvm() {
        Class eagleClass = Eagle.class;   - It will return
            // All public method of this class as well as SuperClass -
        Method[] methods = eagleClass.getMethods();
        for(Method method : methods) {
            sout("Method name : " + method.getName());
            sout("Return type : " + method.getReturnType());
            sout("Class Name : " + method.getDeclaringClass());
            sout("*****");
        }
                                    All public & private methods of Eagle class only
        Method[] methods1 = eagleClass.getDeclaredMethods();
        for(Method method : methods1) {
            sout("MethodName : " + method.getName());
        }
    }
}
```

→ Object class is superclass of all class by default

Output:

→ Method name : fly
Return type : void
ClassName : class Eagle
*****
Method name : wait
Return type : void
className : java.lang.Object

MethodName : fly
MethodName : eat

Invoking / calling Method using Reflection:

```
public class Eagle {
    Eagle() { }
    public void fly(int intpara, boolean boolpara,
                    String strparam) {
        sout("fly intparam: " + intpara + " boolparam: "
             + boolpara + " strparam: " + strparam);
    }
}
public class Main {
    psvm() throws ClassNotFoundException. {
        Class eagleclass = Class.forName("Eagle");
        Object eagleObj = eagleclass.newInstance(); // return obj of Eagle class.
        Method flyMethod = eagleclass.getMethod("fly", int.class,
                           boolean.class, String.class);
        flyMethod.invoke(eagleObj, 1, true, "hello");
    }
}
```

Output: fly intparam: 1 boolparam: true strparam: hello

Reflection of fields

ex: Assume we have same Eagle class used in prev. ex.

```
public class Main {
    psvm() {
        Class eagleClass = Eagle.class;
        // getFields() returns all the public fields

        Field[] fields = eagleClass.getFields();
        for(Field field : fields) {
            sout("FieldName: " + field.getName());
            sout("Type: " + field.getType());
            sout("Modifier: " + Modifier.toString(
                                field.getModifiers()));
            sout("*****");
        }

        // getDeclaredFields() → return all public & private
        // - fields of Eagle class.

        Field[] fields1 = eagleClass.getDeclaredFields();
        for(Field field : fields1) {
            sout("FieldName: " + field.getName());
            sout("Type: " + field.getType());
            sout("Modifier: " + Modifier.toString(
                        field.getModifiers()));
            sout("*****");
        }
    }
}
```

output:
FieldName : breed
Type : java.lang.String
Modifier: public
*****

FieldName : breed
Type: java.lang.String
Modifier: public
*****
FieldName : canSwim
Type: ~~java.lang~~ boolean
Modifier: private
*****

Setting value of public and private field (? will private field can access directly let's see 😊)

```
public class Main {
    psvm() throws NoSuchFieldException {
        Class eagleClass = Eagle.class;
        Eagle eagleObj = new Eagle();
        //get both public & private field with this...
        try {
            Field fpublic = eagleClass.getDeclaredField("breed");
            Field fprivate = eagleClass.getDeclaredField("canSwim");
            fpublic.set(eagleObj, "eagleBrownBreed");
            Sout(eagleObj.breed); // <- word
Error X -->  fprivate.set(eagleObj, true); //throws Exception
            Sout("private field changed");
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
}
```

output

eagleBrownBreed

error: IllegalAccessException due to access of private field directly

Setting the value of private field: (Hack)

```
public class Main{
    psvm(){
        Class eagleclass = Eagle.class;
        Eagle eagleObj = new Eagle();

        Field ffield = eagleClass.getDeclaredField("canSwim");
        ffield.setAccessible(true); //now we can access private field
        ffield.set(eagleObj, true);
        if(ffield.getBoolean(eagleObj)){
            sout("value is set to true");
        }
    }
}
```

giving right to access private field

Output: value is set to true

Note: - Through Reflection we can access private field that is one of the disadvantage that there is no control on code and also breaks OOps encapsulation principle (only with getters & setters we can access private field) but with reflection we can change private value.

Reflection of Constructor

→ Again with reflection we can access private/public constructor and initialize object as well as access all instance variable. which breaks the Singleton pattern rule.

Normally we cannot create an obj from private constructor, but we did it with the help of Reflection of constructor. again breaks OOps principle.

Ex:

```
public class Eagle {

    private Eagle(){ }  // private cons, means we
                        // do not initialize obj of Eagle.
    public void fly(){  // without any obj we can't
        sout("fly");        access instance method.
    }
}
// But with the help of Reflection of Constructor :..
public class Main {
  psvm() throws InstantiationException {

    Class eagleClass = Eagle.class;
    // now access private constructor too.
    Constructor[] eagleConstructorList =
                eagleClass.getDeclaredConstructors();

    for(Constructor eagleConstructor : eagleConstructorList){
      sout("Modifier: " + Modifier.toString(
                        eagleConstructor.getModifiers()));
      eagleConstructor.setAccessible(true);
      Eagle eagleObj = (Eagle) eagleConstructor.
                                newInstance();

      eagleObj.fly();
    }
  }
}
```

Output: Modifier: private
fly

Reflection → is not Encourage
- Inc complexity
- is slow

Through Reflection you break Singleton as Singleton also has private const only). and only obj is created